

Airflow at the center of your pipelines

Airflow Fear: The Hidden Cost of Complexity 

The context

Are you tired of managing heavy infrastructure just to run a few Python scripts? Airflow is great for "The Big Data Era," but in the modern "Efficiency Era," it's often overkill. Why spin up Docker containers, Redis, and Postgres just for a DAG?

Why engineers shift to wpipe

- < 50MB RAM: Run it on a toaster (or a tiny Lambda).
- Pure Python: No YAML hell. Just use the `@state` decorator and focus on your logic.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="task", retry_count=3, retry_delay=2)
def task(data):
    return {"ok": run(data["job"])}

pipe = Pipeline(pipeline_name="airflow_like", tracking_db="jobs.db")
pipe.set_steps([task])

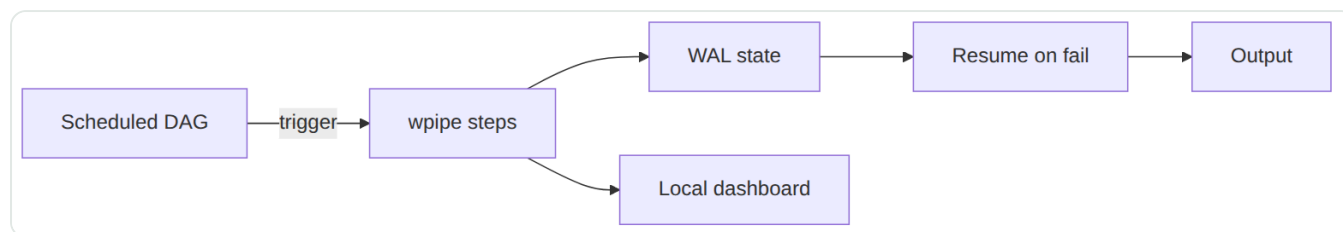
# No scheduler daemon, no heavy metadata DB.
```

At a glance

Aspect	Legacy / Airflow	wpipe
Definition	Config/DSL or canvas	Python code
State	Scattered / none	SQLite WAL checkpoints
Observability	Partial	Tracker SQL + dashboard
Footprint	Heavy	<50MB RAM

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads

A community of efficiency-first developers.

<50MB RAM

Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL

Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Airflow** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.